CHAINGUARD: COST-OPTIMIZED BLOCKCHAIN BASED CLOUD FILE INTEGRITY VERIFICATION

A PROJECT REPORT

Submitted by

SHARMILEE B	**2116231901049**
SOMILA SA	**2116231901052**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled **"CHAINGUARD: COST-OPTIMIZED BLOCKCHAIN-BASED CLOUD FILE INTEGRITY VERIFICATION"** is the Bonafide work of "**SHARMILEE B (2116231901049), SOMILA SA (2116231901052)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
Dr. P. Tamilselvi, M.E., Ph.D.,
HEAD OF DEPARTMENT
PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE
Mrs. Divya R
SUPERVISOR
ASSISTANT PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Cloud storage provides scalable and convenient access to digital files, but ensuring the integrity of stored data remains a significant security concern. Traditional cloud storage systems rely heavily on service providers for data management and integrity assurance. Blockchain technology provides tamper-evident record keeping but storing individual file hashes as separate blockchain transactions can increase transaction overhead and operational cost. Existing Merkle Tree-based approaches reduce the amount of data required for verification, but fixed batching strategies may not respond effectively to changing workloads and blockchain conditions. This project proposes ChainGuard, an adaptive Merkle Root batching framework for blockchain-based cloud file integrity verification. The system generates SHA-256 hashes for uploaded files and organizes them into batches using a Merkle Tree. Instead of storing individual file hashes on the blockchain, only the Merkle Root of each batch is recorded through a smart contract. Individual files can subsequently be verified using their hashes and corresponding Merkle proofs. The proposed adaptive batching mechanism considers factors such as batch size, file upload rate, waiting time, and gas price to determine an appropriate blockchain commitment decision. The system is designed using React.js, Node.js, Express.js, Solidity, Hardhat, and ethers.js. Its performance will be evaluated against individual commitment, fixed batch-size, and fixed time-window strategies using transaction count, gas consumption, verification latency, throughput and storage overhead. ChainGuard aims to balance blockchain cost and integrity-verification performance while providing a transparent and tamper-evident verification mechanism for cloud-stored files.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Mrs. R. Divya, Assistant professor** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

B. SHARMILE (2116231901049)

SA. SOMILA (2116231901052)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vi
	LIST OF FIGURES	vi
1	**INTRODUCTION**	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES	2
	1.4 SCOPE	2
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	3
	1.6 SDG ALIGNMENT	3
2	**LITERATURE REVIEW**	4
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	4
	2.2 RESEARCH GAPS	5
	2.3 PROPOSED SYSTEM	5
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	5
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	6
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	6
3	**SYSTEM DESIGN**	8
	3.1 SYSTEM ARCHITECTURE	8
	3.2 MODULE DESIGN	9
	3.3 DATA FLOW / UML DIAGRAMS	10
	3.4 THREAT MODEL AND ATTACK SURFACE	11
	3.5 SECURITY CONTROLS	11
4	**SYSTEM DESCRIPTION**	16
	4.1 DATA COLLECTION	16
	4.2 DATA PREPROCESSING	17
	4.2.1 ALGORITHMS / METHODOLOGY	17
	4.2.2 RESULTS AND DISCUSSION	20
5	**CONCLUSION**	22
	REFERENCES	23
	APPENDIX	24

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	Software Requirements	9
3.2	ChainGuard System Modules	9
4.1	Threat Model and Attack Surface	10
4.1	Security Controls	11
4.3	Performance Evaluation Metrics	20
4.4	Comparison of Fixed and Adaptive Batching	21

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	System Architecture of ChainGuard	10
3.2	Data Flow Diagram of ChainGuard	12
3.3	Use Case Diagram	13
3.4	Activity Diagram for File Upload and Verification	14
3.5	Sequence Diagram for Merkle Root Generation and Blockchain Commitment	15

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Cloud computing has become an essential technology for storing, managing, and accessing digital data. It provides users with scalable storage, remote accessibility, and reduced infrastructure requirements. However, storing files on third-party cloud platforms introduces concerns regarding unauthorized modification, accidental data corruption, and the ability to independently verify the integrity of stored information. Traditional integrity verification methods often depend on centralized mechanisms or trusted third-party auditors. Although these approaches can identify data changes, they may introduce additional trust requirements and operational overhead. Blockchain technology provides a distributed and tamper-evident ledger that can be used to maintain integrity records. However, storing every file hash directly on a blockchain can result in a large number of transactions and increased gas consumption. A Merkle Tree is a cryptographic data structure that combines multiple file hashes into a single Merkle Root. This enables the verification of individual files through Merkle Proofs without requiring every file hash to be stored on chain. ChainGuard combines SHA-256 hashing, Merkle Trees and blockchain smart contracts to provide a file integrity verification framework. It further proposes an adaptive batching mechanism that considers workload and blockchain conditions to decide when Merkle Roots should be committed. The objective is to reduce blockchain transaction overhead while maintaining acceptable verification latency and system performance.

1.2 PROBLEM STATEMENT

Cloud storage systems require reliable mechanisms to verify that stored files have not been modified or corrupted. Centralized integrity verification can create dependence on the cloud service provider, while blockchain-based verification can introduce high transaction overhead when individual file hashes are stored separately.

Although Merkle Trees reduce on-chain storage requirements by aggregating file hashes, fixed batch-size and fixed time-window approaches may not efficiently handle variations in file upload rates, batch accumulation and gas prices.

Therefore, a cost-aware and adaptive file integrity verification framework is required to determine suitable blockchain commitment times while preserving the integrity and

1.3 OBJECTIVES

The primary objective of ChainGuard is to develop an adaptive Merkle Root batching framework for cost-efficient blockchain-based cloud file integrity verification.

The specific objectives are:

- To generate unique SHA-256 hashes for uploaded files.
- To construct Merkle Trees from batches of file hashes and generate Merkle Roots.
- To store Merkle Roots on a blockchain using smart contracts.
- To verify individual files using Merkle Proofs and detect unauthorized modifications.
- To develop an adaptive batching strategy based on batch size, upload rate, waiting time, and gas price.
- To compare adaptive batching with individual, fixed batch-size and fixed time-window commitment strategies.
- To evaluate transaction count, gas consumption, verification latency, throughput and storage overhead.

1.4 SCOPE

The scope of ChainGuard includes:

- Secure file hashing using SHA-256.
- Batch-based aggregation of file hashes using Merkle Trees.
- Smart contract-based storage and retrieval of Merkle Roots.
- Individual file integrity verification using Merkle Proofs.
- Adaptive decisions for blockchain commitment based on workload and gas conditions.
- Experimental comparison of different batching strategies using a local blockchain environment.

The initial implementation focuses on a prototype cloud file integrity verification system and local blockchain testing. Large-scale public blockchain deployment, production-grade cloud infrastructure, advanced user access management and machine-learning-based gas prediction may be considered future enhancements.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

ChainGuard is designed to meet the following security requirements.

Requirement	Description
Integrity	Detect unauthorized changes to uploaded files using cryptographic hashes.
Authenticity of commitment	Use blockchain transactions to establish an auditable Merkle Root commitment.
Tamper evidence	Prevent previously committed batch records from being overwritten by the smart contract.
Verifiability	Support individual file verification using Merkle Proofs.
Availability	Maintain access to file metadata and verification functions when the application and blockchain are available.
Confidentiality	Keep actual file content off-chain and apply appropriate access controls to file storage.
Auditability	Maintain transaction identifiers and timestamps for committed Merkle Roots.
Secure communication	Use secure communication channels and protect application credentials and blockchain signing keys.

Blockchain-based integrity verification does not guarantee file confidentiality, secure access control or availability of the cloud storage service. These requirements must be addressed through additional application and infrastructure controls.

1.6 SDG ALIGNMENT

ChainGuard supports the following United Nations Sustainable Development Goals.

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9 – Industry, Innovation and Infrastructure	Target 9.1: Develop reliable, sustainable and resilient infrastructure.	Develops a blockchain-based file integrity framework that supports reliable digital data verification.

2	SDG 16 – Peace, Justice and Strong Institutions	Target 16.6: Develop effective, accountable and transparent institutions.	Provides tamper-evident records and transparent verification of file integrity commitments.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Existing cloud data integrity verification systems use cryptographic hashing, Merkle Trees, third-party auditing and blockchain-based verification to detect unauthorized modifications and establish trustworthy integrity records.

1. Traditional Cloud Integrity Verification

Traditional systems use cryptographic hash functions to generate a digest of a file. The digest is later recalculated and compared with the previously stored value to detect modifications. These approaches are computationally simple but may rely on a centralized storage provider to maintain trusted reference hashes.

2. Third-Party Auditing

Third-party auditing mechanisms allow an independent auditor to verify cloud-stored data on behalf of users. Cryptographic authentication structures, including Merkle Hash Trees, can support efficient verification. However, reliance on an external auditor introduces additional trust and communication requirements.

3. Blockchain-Based Integrity Verification

Blockchain-based approaches store integrity-related information on a distributed ledger. Smart contracts can automate the recording and retrieval of verification commitments. Blockchain provides tamper-evident records, but transaction fees, confirmation of latency and limited on-chain storage can create scalability challenges.

4. Merkle Tree-Based Verification

Merkle Trees combine multiple file hashes into a hierarchical structure and generate a single Merkle Root. Individual file integrity can be verified using an authentication path, known as a Merkle Proof. This reduces the amount of information required for individual verification

compared with processing every hash in the batch.

5. Fixed Batching Approaches

Fixed batching combines a predetermined number of file hashes or waits for a predefined time interval before committing a Merkle Root. While batching can reduce the number of blockchain transactions, a fixed configuration may not be suitable for both low-volume and high-volume workloads.

Comparison of existing approaches:

Approach	Security Mechanism	Advantage	Limitation
Traditional hashing	Cryptographic hash	Simple integrity checking	Centralized reference-hash dependency
Third-party auditing	Cryptographic proofs	Independent verification	Auditor trust and communication overhead
Blockchain auditing	Distributed ledger and smart contracts	Tamper-evident audit records	Transaction and gas overhead
Merkle Tree verification	Hash tree and Merkle Proof	Efficient individual verification	Does not alone determine optimal commitment timing
Fixed batching	Batch-size or time threshold	Reduces transaction frequency	Limited workload and gas adaptability

2.2 RESEARCH GAPS

The reviewed research provides important foundations for cloud data integrity, Merkle-based authentication, blockchain auditing, and transaction optimization. However, several challenges remain relevant to the ChainGuard problem.

Fixed batching limitations: Predetermined batch sizes and time windows may not adapt effectively to changing file upload rates.

Transaction overhead: Blockchain commitments may introduce additional cost when the batching strategy is not suited to the current workload.

Latency and cost trade-off: Increasing batch size can reduce the number of blockchain transactions but may increase the time a file waits for commitment. Gas-price variation: Blockchain transaction costs can change with network conditions, affecting the cost efficiency of commitment decisions.

Integrated evaluation: A common experimental comparison of individual, fixed batch-size, fixed time-window and adaptive Merkle Root commitment strategies is needed for the selected workload and implementation environment.

Identified research gap: There is scope to investigate an adaptive Merkle Root commitment strategy that uses workload and blockchain conditions to balance transaction overhead, gas consumption, and file verification performance.

This gap is the motivation for the proposed ChainGuard framework. The final research-gap statement should be validated against the exact findings and limitations of all selected literature references before submission.

2.3 PROPOSED SYSTEM

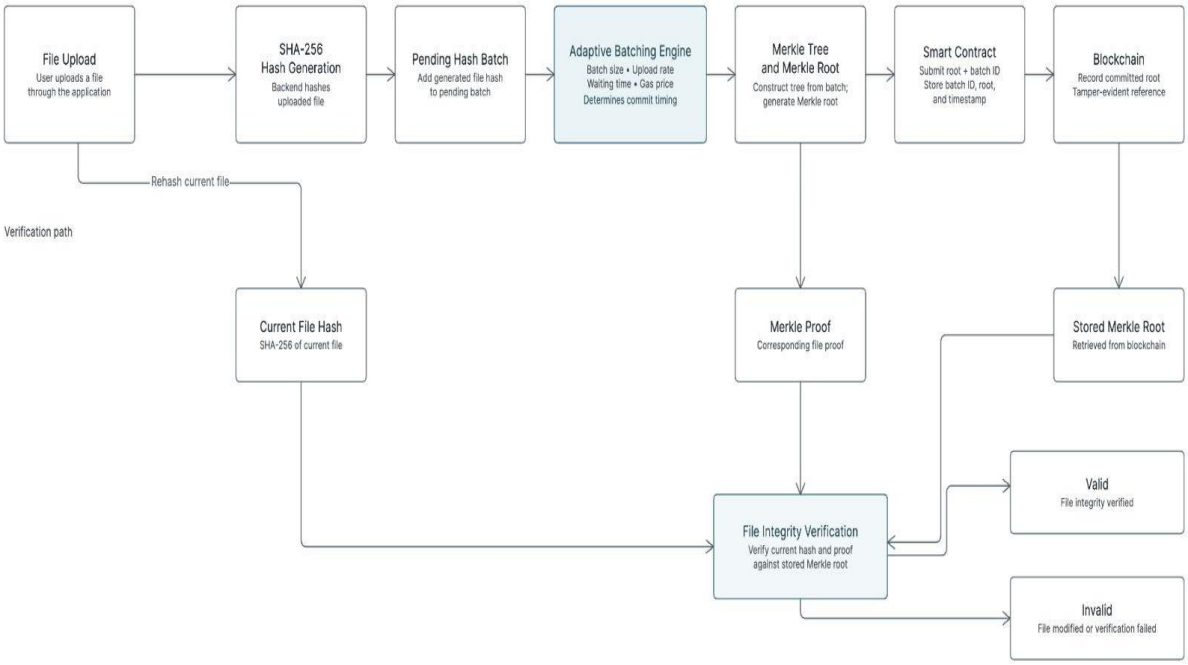
2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

ChainGuard is a blockchain-based cloud file integrity verification framework that combines SHA-256 hashing, Merkle Tree aggregation and smart contract-based Merkle Root storage.

When a user uploads a file, the system generates its SHA-256 hash and adds it to a pending batch. The hashes in the batch are combined to construct a Merkle Tree, from which a Merkle Root is generated.

The proposed adaptive batching mechanism evaluates factors such as the current batch size, upload rate, elapsed waiting time and gas price. Based on the defined decision rules, the system determines whether the Merkle Root should be committed to the blockchain or whether batching should continue, subject to a maximum waiting-time constraint.

The smart contract stores the Merkle Root against a unique batch ID along with a timestamp. It prevents an existing batch record from being overwritten. During verification, the system recalculates the file hash, uses its Merkle Proof and checks whether the proof leads to the committed root.



2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- To provide cryptographic integrity verification for cloud-stored files.
- To reduce on-chain data storage by recording only Merkle Roots.

- To automate blockchain commitments using smart contracts.
- To introduce adaptive batching based on workload and gas conditions.
- To reduce unnecessary blockchain transactions without exceeding the configured waiting-time limit.

- To support individual file verification through Merkle Proofs.
- To evaluate the trade-off between blockchain cost, verification latency and throughput.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

Reduced on-chain data: Only Merkle Roots are stored on the blockchain instead of individual file hashes.

Efficient verification: Merkle Proofs enable verification of an individual file without reconstructing the entire batch.

Tamper-evident records: Blockchain commitments provide a persistent record of the committed Merkle Roots.

Adaptive commitment: Batching decisions can respond to workload and gas-price conditions.

Lower transaction overhead potential: Aggregating file hashes can reduce the number of blockchain transactions.

Modular design: File handling, hashing, batching, verification, and blockchain integration are separated into independent modules.

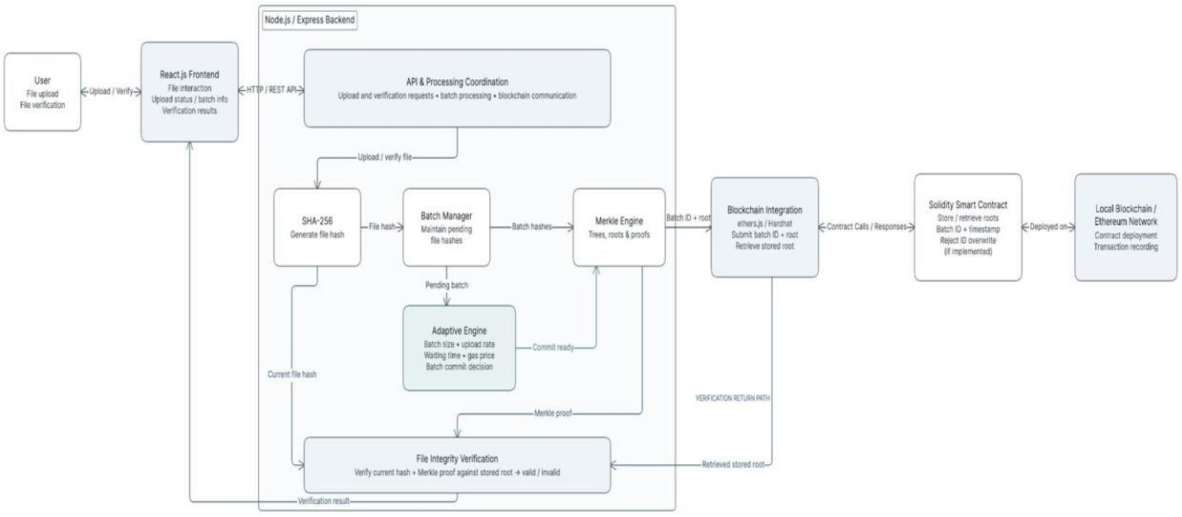
Auditable operation: Batch identifiers, timestamps and transaction hashes support verification and traceability.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

ChainGuard follows a modular client-server architecture that integrates file processing, cryptographic operations, adaptive batching and blockchain interaction.

The frontend provides the interface for uploading files and initiating verification. The Node.js backend handles file processing, hash generation, batch management and Merkle Tree construction. The adaptive batching engine determines when a Merkle Root should be submitted. The blockchain module communicates with the smart contract using ethers.js.



The architecture is divided into the following layers:

- Presentation layer: Provides the user interface for uploading and verifying files.
- Application layer: Handles requests, file processing, hash generation and batch management.
- Adaptive batching layer: Evaluates batching conditions and decides when to commit a Merkle Root.
- Blockchain layer: Uses smart contracts to store and retrieve batch commitments.
- Verification layer: Validates individual file hashes against the corresponding Merkle Root using Merkle Proofs.

3.2 MODULE DESIGN

Module	Description
User Interface	Allows users to upload files, view batches and initiate verification.
File Processing	Accepts files and processes their contents for hashing.
SHA-256 Hashing	Generates a cryptographic digest for each file.
Batch Management	Collects file hashes in a pending batch and maintains batch metadata.
Merkle Tree Generation	Combines file hashes into a Merkle Tree and generates the root.
Adaptive Batching	Evaluates batch size, upload rate, waiting time and gas conditions to determine commitment timing.
Smart Contract	Stores the Merkle Root and timestamp for each unique batch ID and provides retrieval functionality.
Blockchain Integration	Connects the backend to the blockchain using ethers.js and submits transactions.
File Verification	Recalculates the file hash and verifies its Merkle Proof against the committed root.
Performance Evaluation	Collects measurements for transaction count, gas, latency, throughput and storage overhead.

SOFTWARE REQUIREMENTS

Software	Purpose
React.js	Frontend development
Node.js	Backend runtime
Express.js	REST API development
Solidity	Smart contract development
Hardhat	Local blockchain development and contract testing
ethers.js	Backend-to-blockchain interaction
SHA-256	File hash generation
Merkle Tree library	Merkle Root and Merkle Proof generation
Visual Studio Code	Development environment
Git	Source-code version control

3.4 THREAT MODEL AND ATTACK SURFACE

The threat model identifies possible attacks against file integrity, batching, blockchain interaction, and application security.

Threat	**Attack Surface**	**Potential Impact**
Unauthorized file modification	Cloud storage or file upload	Integrity violation
Hash substitution	Backend or file-processing module	Incorrect integrity reference
Merkle Proof manipulation	Verification API	False verification attempt
Batch ID collision or reuse	Batch manager / smart contract	Incorrect batch association
Unauthorized contract calls	Blockchain interface	Unapproved commitment attempts
Transaction replay or duplicate submission	Backend and blockchain interface	Repeated or inconsistent transactions processing
Private-key exposure	Backend environment	Unauthorized blockchain transactions
Denial of service	Upload and verification APIs	Reduced availability
Malicious or oversized uploads	File-processing module	Resource exhaustion
Gas-price fluctuation	Blockchain network	Increased commitment cost or delay

The primary assets are the uploaded files, their cryptographic hashes, batch metadata, Merkle Roots, Merkle Proofs, blockchain signing credentials, and verification results.

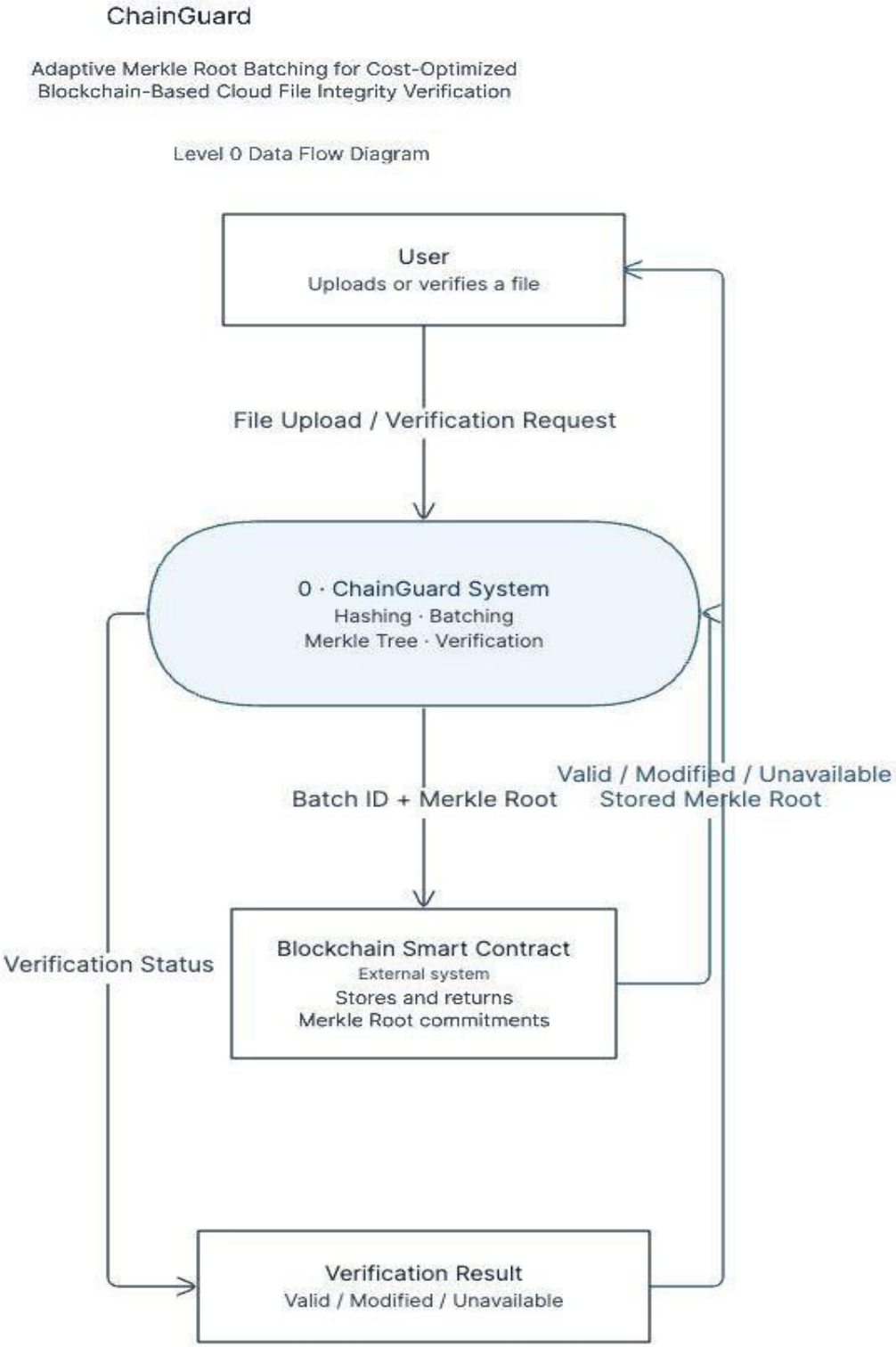
The system assumes that the selected hash algorithm is secure for practical integrity checking, that the smart contract is correctly implemented, and that the blockchain network provides the expected transaction finality. A compromised backend or signing key may still submit incorrect commitments, so blockchain immutability alone cannot guarantee that the initial data submitted was trustworthy.

3.5 SECURITY CONTROLS

Security Control	Implementation / Purpose
SHA-256 hashing	Detects changes in file content.
Merkle Proof verification	Verifies an individual file against its batch commitment.
Immutable batch records	Prevents overwriting an existing batch ID in the smart contract.
Input validation	Validates batch of IDs, file types, sizes, and API parameters.
Private-key protection	Keeps signing credentials in environment configuration and excludes secrets from source control.
Access control	Restricts file and API operations to authorized users when authentication is implemented.
Secure communication	Uses HTTPS in production deployment.
Error handling	Prevents failed transactions or verification requests from being reported as successful.
Event logging	Supports auditing of blockchain commitment activity.
Resource limits	Helps reduce denial-of-service risks from excessive or oversized uploads.

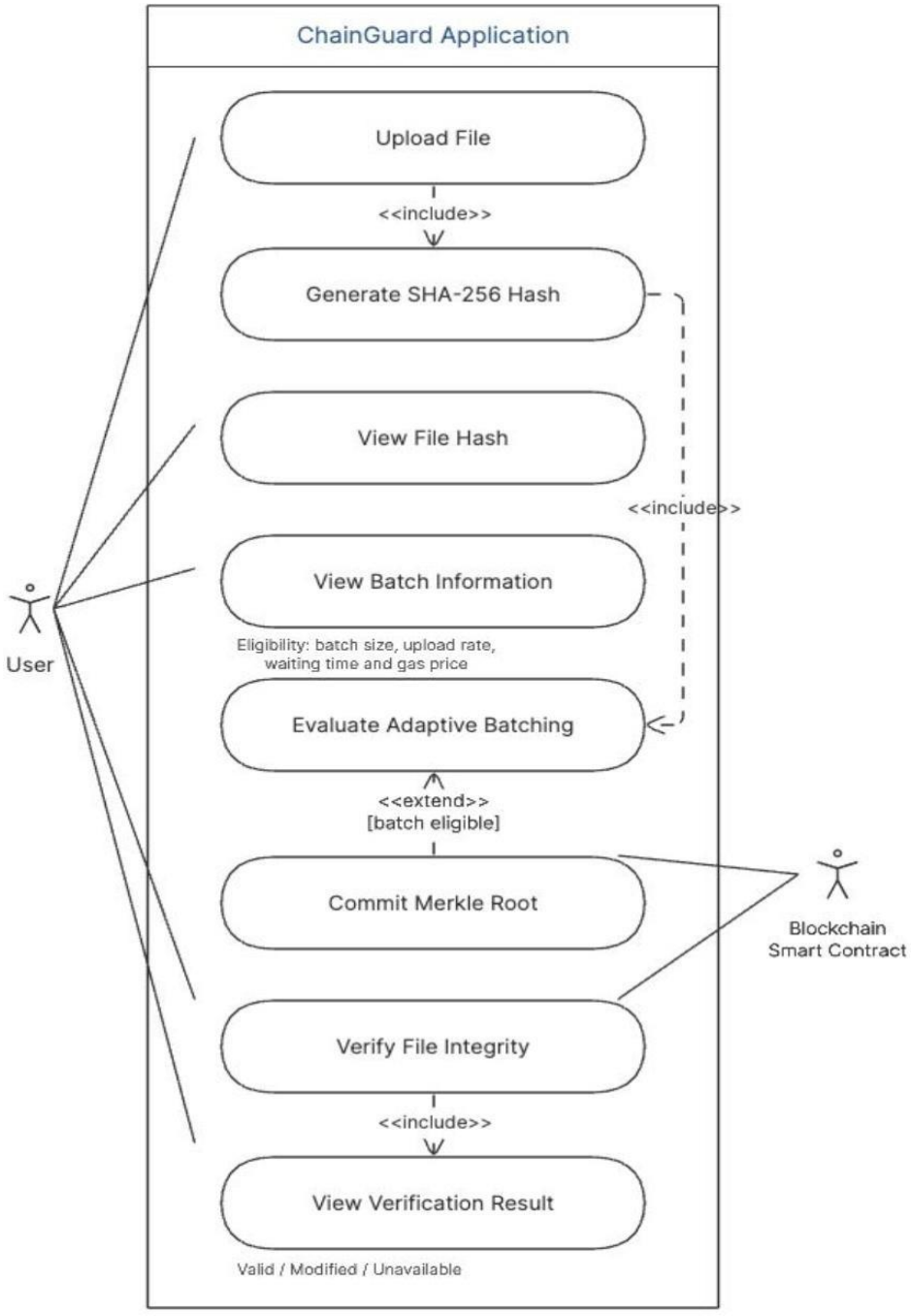
3.3 DATA FLOW / UML DIAGRAMS

3.3.1 Data Flow Diagram

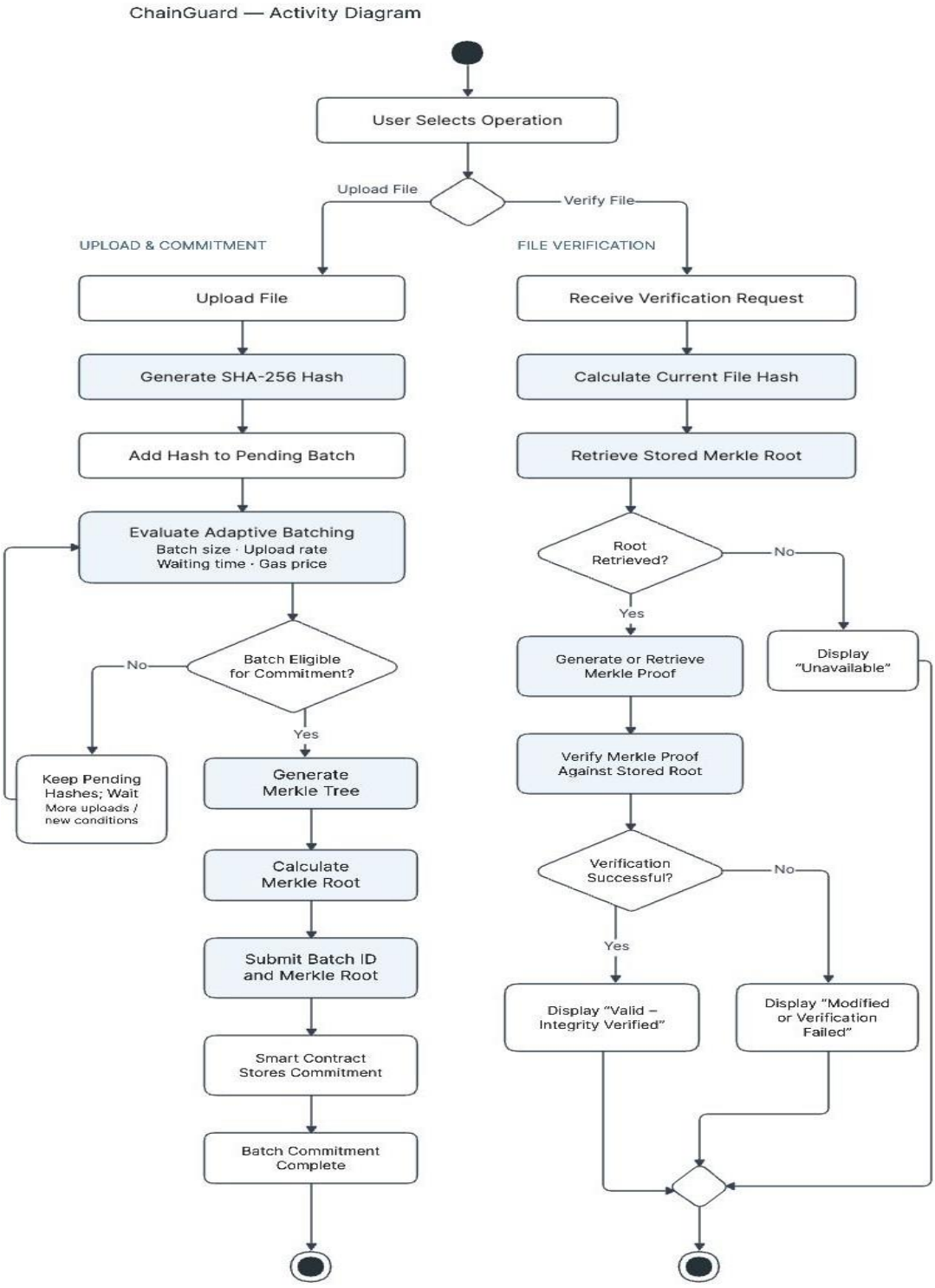


3.3.2 Use Case Diagram

The main actor is the user, who interacts with the ChainGuard application to upload files, view generated hashes and batch information, commit Merkle Roots when eligible, and verify stored files.



3.3.3 Activity Diagram



CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

ChainGuard does not require a public medical, financial or image dataset. The system uses file-based test inputs and generated workloads to evaluate the integrity verification and adaptive batching mechanisms.

The test inputs consist of files of different sizes and formats, along with controlled upload patterns to simulate different workload conditions. Each uploaded file is processed to generate a SHA-256 hash. The resulting hashes are used to construct Merkle Trees and produce batch Merkle Roots.

The experimental workload is designed to evaluate the performance of different commitment strategies under varying numbers of files, upload rates and batching conditions.

The following data will be collected during testing:

- File identifier and file size.
- SHA-256 hash of each uploaded file.
- Batch identifier and batch size.
- Merkle Root generated for each batch.
- Time taken to generate hashes and Merkle Proofs.
- Blockchain transaction hash and transaction status.
- Gas consumed by each blockchain commitment.
- File verification result and verification latency.
- Batch waiting time and total processing time.

Experimental Workload

Parameter	**Description**
File count	Number of files submitted during each experiment
File size	Size of each input file
Upload rate	Number of files submitted per unit of time
Workload pattern	Low, medium, high and burst uploads

Batch strategy	Individual, fixed-size, fixed-time and adaptive
Blockchain environment	Local Hardhat development network
Repetitions	Multiple runs for each experimental configuration

The workload sizes and number of repetitions should be finalized based on the system's available resources and the time required for each run.

4.2 DATA PREPROCESSING

Data preprocessing prepares the uploaded files and generated metadata for hashing, batching, blockchain commitment, and verification.

The preprocessing steps are:

- **File validation:** Check that the uploaded file is present and meets the configured size and format restrictions.
- **File identification:** Assign a unique identifier to each file to maintain its association with the generated hash and batch.
- **Hash generation:** Process the file contents using SHA-256 to generate a fixed-length cryptographic digest.
- **Hash representation:** Convert the digest into a consistent representation before adding it to the Merkle Tree.
- **Batch formation:** Add the generated hashes to a pending batch according to the configured batching strategy.
- **Merkle Tree construction:** Combine the hashes using the selected Merkle Tree implementation to produce a Merkle Root.
- **Metadata preparation:** Associate the batch ID, file identifiers, Merkle Root and relevant timestamps.
- **Verification preparation:** Generate or retrieve the Merkle Proof required for individual file verification.

Consistent hashing and Merkle Tree construction rules are essential. The same input hash representation, leaf ordering, pair-combination rule and odd-leaf handling must be used during both tree generation and verification.

4.2.1 ALGORITHMS / METHODOLOGY

The ChainGuard methodology consists of file hashing, Merkle Tree construction, adaptive batching, blockchain commitment and integrity verification.

Algorithm 1: SHA-256 Hash Generation

Input: Uploaded file F Output:
File hash H

1. Receive the uploaded file F.
2. Validate the file.
3. Read the file contents.
4. Apply SHA-256 to the contents.

5. Generate the cryptographic digest H.
6. Associate H with the file identifier.
7. Return H.

The SHA-256 hash acts as a compact representation of the file contents. When a file is modified, its recalculated hash is expected to differ from the original hash.

Algorithm 2: Merkle Tree Construction

Input: File hashes H = {h1, h2, ..., hn} Output:
Merkle Root R

1. Create the leaf nodes from the input hashes.
2. Arrange the leaf nodes in a defined order.
3. Pair adjacent nodes.
4. Hash each pair to generate parent nodes.
5. If a level contains an unpaired node, apply the defined odd-node handling rule.
6. Repeat until a single root node remains.
7. Return the final node as the Merkle Root R.

The Merkle Root represents the hashes of all files included in the batch. Merkle Proofs provide the sibling hashes required to verify an individual leaf against this root.

Algorithm 3: Adaptive Merkle Root Batching

The adaptive batching mechanism evaluates the current batch size, upload rate, waiting time and gas price to determine when to submit a Merkle Root.

Let:
B = current number of file hashes in the batch. R = observed file upload rate.
W = elapsed batch waiting time. G = current gas price.
B_{max} = configured maximum batch size. W_{max} = configured maximum waiting time.

Input: B, R, W, G Output: Commit or Wait

1. Receive the current batch status.
2. Calculate or update the upload rate.
3. Read the elapsed waiting time.
4. Obtain the current gas-price information.

5. If the batch is empty: Return WAIT.
6. If B >= Bmax: Return COMMIT.
7. If W >= Wmax: Return COMMIT.
8. Evaluate the adaptive decision rule using upload rate and gas-price conditions.
9. If the commit condition is satisfied: Return COMMIT.
10. Otherwise: Return WAIT.

The maximum batch size and maximum waiting time act as safety limits. The adaptive decision rule must be defined precisely in the implementation, including its thresholds, normalization if used, and behavior when gas-price information is unavailable.

For research evaluation, the adaptive strategy should use clearly documented parameter values and the same test workloads as the baseline strategies. This makes the comparison reproducible.

Algorithm 4: Blockchain Commitment

Input: BatchID, MerkleRoot Output:
Transaction status and hash

1. Receive the eligible batch ID and Merkle Root.
2. Validate the batch ID and root.
3. Check whether the batch has already been committed.
4. If the batch already exists:
   Reject the duplicate commitment.
5. Otherwise:
   Call storeMerkleRoot(batchId, root).
6. Submit the transaction to the blockchain.
7. Wait for the transaction receipt.
8. Record the transaction hash and status.
9. Return the commitment details.

The ChainGuard smart contract is designed to store each Merkle Root against a unique batch ID and record its timestamp. The getMerkleRoot(batchId) function retrieves the committed root for later verification.

Algorithm 5: File Integrity Verification

Input: File F, Batch ID, Merkle Proof Output: Verification result

1. Receive the file and its batch information.
2. Calculate the current SHA-256 hash H.
3. Retrieve the stored Merkle Root for the batch.

4. Obtain the Merkle Proof associated with H.
5. Reconstruct the root using H and the proof.
6. Compare the reconstructed root with the stored Merkle Root.
7. If both roots match:
   Return VALID.
8. Otherwise:
   Return INVALID / MODIFIED.

The verification process establishes whether the submitted file matches a leaf included in the committed Merkle Tree, provided that the correct batch association and proof are used.

4.2.2 RESULTS AND DISCUSSION

The performance evaluation of ChainGuard is designed to compare the proposed adaptive batching strategy with individual commitment, fixed batch-size commitment and fixed time-window commitment.

The experiments will use identical workloads and a consistent local blockchain configuration for all strategies. The performance measurements will be collected over repeated runs to account for execution variability.

Performance Evaluation Metrics:

Metric	Measurement	Purpose
Transaction Count	Total blockchain transactions	Measures blockchain transaction overhead
Gas Consumption	Gas used by commitment transactions	Measures computational cost on-chain
Verification Latency	Time taken to verify a file	Measures integrity-checking responsiveness
Throughput	Number of files processed per second	Measures processing capacity
Storage Overhead	On-chain data and supporting metadata	Measures storage requirements
Batch Waiting Time	Time from entering a batch to commitment	Measures commitment delay
Hash Generation Time	Time required to generate file hashes	Measures cryptographic processing overhead

Strategy	Commitment Condition	Expected Characteristic
Individual Commitment	Commit each file separately	Simple commitment with potentially high transaction count
Fixed Batch-Size	Commit after a fixed number of files	Predictable batching, but less responsive to workload changes
Fixed Time-Window	Commit after a predefined time interval	Predictable time-based commitment, but batch sizes may vary
Adaptive Batching	Commit according to workload, waiting time and gas conditions	Potentially better balance between cost and latency

The transaction count for each strategy can be compared using:

Transaction Reduction (%) = **((Tb - Ta) / Tb) × 100** represents the transaction count of a selected baseline and 'Ta' represents the adaptive strategy transaction count.

Average Verification Latency = **(Σ Li) / n**

where 'Li' is the measured verification latency $\mathbf{L_{avg} = (L1 + L2 + L3 + ... + Ln) / n}$, and n is the total number of verification operations.

Throughput can be calculated as:

Throughput = N / T

Where N is the number of completed file operations and T is the total elapsed time.

To be completed after experiments:

The numerical results, graphs, average values, percentage improvements and conclusions must be inserted after running the actual system. No performance improvement should be claimed before it has been measured.

The discussion should examine whether adaptive batching reduces blockchain transactions and gas consumption while keeping verification latency and batch waiting time within acceptable limits. It should also identify workloads where adaptive batching performs better or worse than the fixed strategies and explain the reasons.

CHAPTER 5
CONCLUSION

ChainGuard proposes an adaptive Merkle Root batching framework for blockchain-based cloud file integrity verification. The system combines SHA-256 hashing, Merkle Tree construction and smart contract-based blockchain commitments to support tamper-evident verification of cloud-stored files.

By storing only Merkle Roots on the blockchain, the framework aims to reduce the need for individual on-chain file hash transactions. The proposed adaptive batching mechanism considers batch size, upload rate, waiting time and gas-price conditions to determine suitable commitment decisions. Merkle Proofs enable individual file verification against the corresponding committed root.

The project integrates frontend file handling, backend processing, cryptographic hashing, Merkle Tree operations and blockchain interaction into a modular architecture. The proposed experimental evaluation compares individual, fixed batch-size, fixed time-window and adaptive batching strategies using transaction count, gas consumption, verification latency, throughput, storage overhead and batch waiting time.

The final effectiveness of the adaptive strategy will be established through experimental results. The project provides a foundation for investigating cost-aware blockchain commitments and efficient integrity verification in cloud storage environments.

Future Enhancements

- Integrating machine-learning-based gas-price prediction into the adaptive batching engine.
- Deploying the system on a public Ethereum test network.
- Integrating cloud storage platforms such as Firebase Storage or Amazon S3.
- Supporting multi-user authentication and role-based access control.
- Evaluating the framework with larger workloads and distributed environments.
- Exploring automated parameter tuning to improve the cost-latency trade-off.

REFERENCES

1. C. Wang, Y. Sun, B. Liu, L. Xue, and X. Guan, “Blockchain-based dynamic cloud data integrity auditing via non-leaf node sampling of rank-based Merkle hash tree,” IEEE Transactions on Network Science and Engineering, vol. 11, no. 5, pp. 3931–3942, Sep./Oct. 2024, doi: 10.1109/TNSE.2024.3393978.

2. Y. Zhang, C. Xu, X. Lin, and X. Shen, “Blockchain-based public integrity verification for cloud storage against procrastinating auditors,” IEEE Transactions on Cloud Computing, vol. 9, no. 3, pp. 923–937, Jul.–Sep. 2021, doi: 10.1109/TCC.2019.2908400.

3. Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, “Enabling public auditability and data dynamics for storage security in cloud computing,” IEEE Transactions on Parallel and Distributed Systems, vol. 22, no. 5, pp. 847–859, May 2011, doi: 10.1109/TPDS.2010.183.

4. Y. Wang, K. Li, Y. Tang, J. Chen, Q. Zhang, X. Luo, and T. Chen, “Towards saving blockchain fees via secure and cost-effective batching of smart-contract invocations,” IEEE Transactions on Software Engineering, vol. 49, no. 4, pp. 2980–2995, Apr. 2023, doi: 10.1109/TSE.2023.3237123.

5. IEEE Conference Papers

6. R. Mars, A. Abid, S. Cheikhrouhou, and S. Kallel, “A machine learning approach for gas price prediction in Ethereum blockchain,” in Proc. IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC), 2021, doi: 10.1109/COMPSAC51774.2021.00033.

7. C. Zhang, C. Xu, J. Xu, Y. Tang, and B. Choi, “GEM2-tree: A gas-efficient structure for authenticated range queries in blockchain,” in Proc. IEEE 35th International Conference on Data Engineering (ICDE), 2019, pp. 842–853, doi: 10.1109/ICDE.2019.00080.

8. D. Lan, H. Wang, C. Yin, L. Zhou, C. Ge, and X. Lu, “Gas price prediction based on machine learning combined with Ethereum mempool,” in Proc. IEEE 19th International Conference on Mobile Ad Hoc and Smart Systems (MASS), 2022, doi: 10.1109/MASS56207.2022.00057.

APPENDIX

APPENDIX A — SMART CONTRACT INTERFACE

The ChainGuard smart contract is designed to store and retrieve Merkle Roots using unique batch identifiers.

Function	Description
storeMerkleRoot(batchId, merkleRoot)	Stores a Merkle Root and timestamp for a new batch ID.
getMerkleRoot(batchId)	Retrieves the stored Merkle Root for a batch.

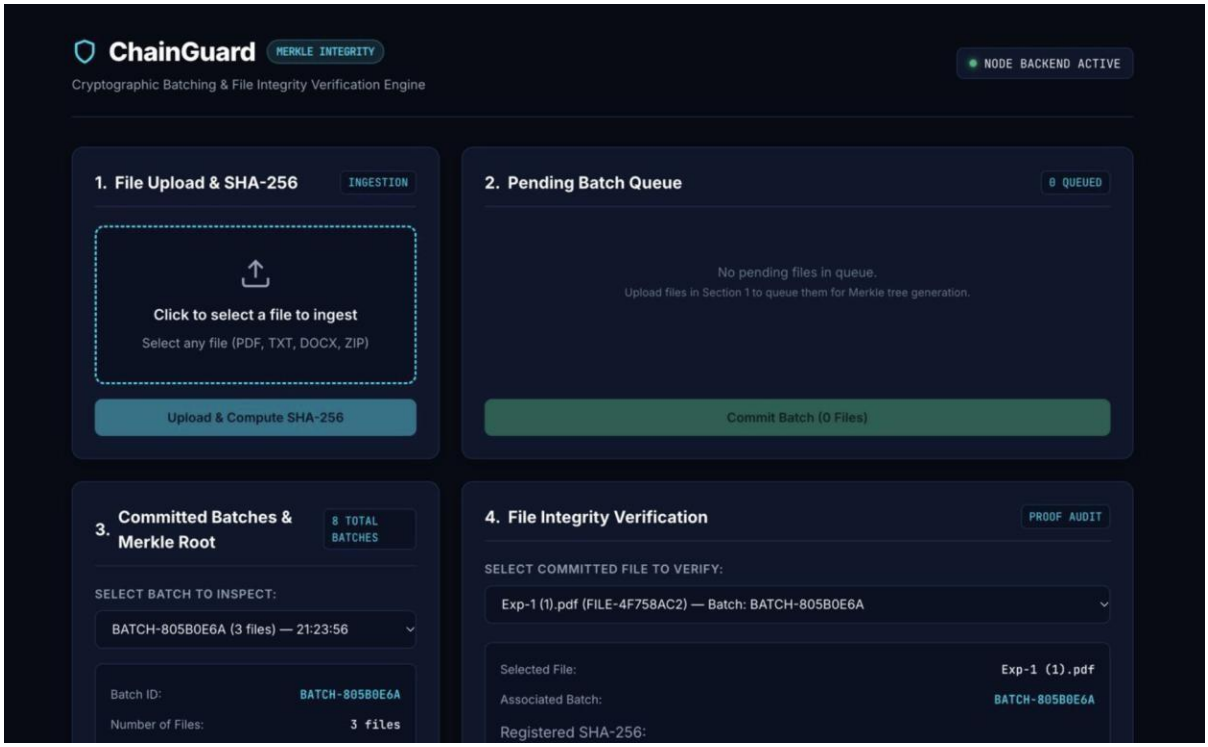
The contract should reject attempts to overwrite an existing batch record and provide suitable validation and error handling.

APPENDIX B — BACKEND API

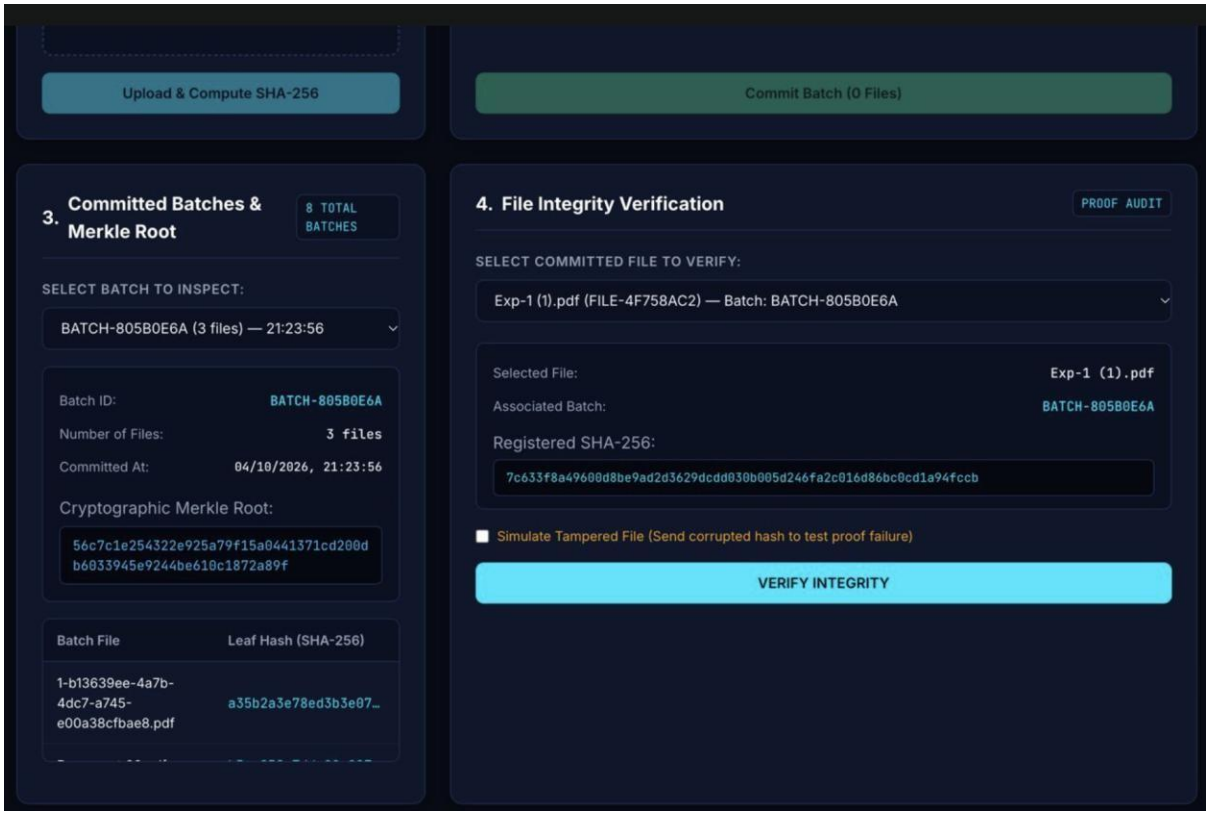
HTTP Method	Endpoint	Purpose
POST	/api/blockchain/commit/:batchId	Commits an eligible batch Merkle Root to the blockchain.
GET	/api/blockchain/batch/:batchId	Retrieves the committed Merkle Root for a batch.

The commitment endpoint should return the batch ID, Merkle Root, transaction hash, and transaction status. The retrieval endpoint should return the stored root and relevant batch details.

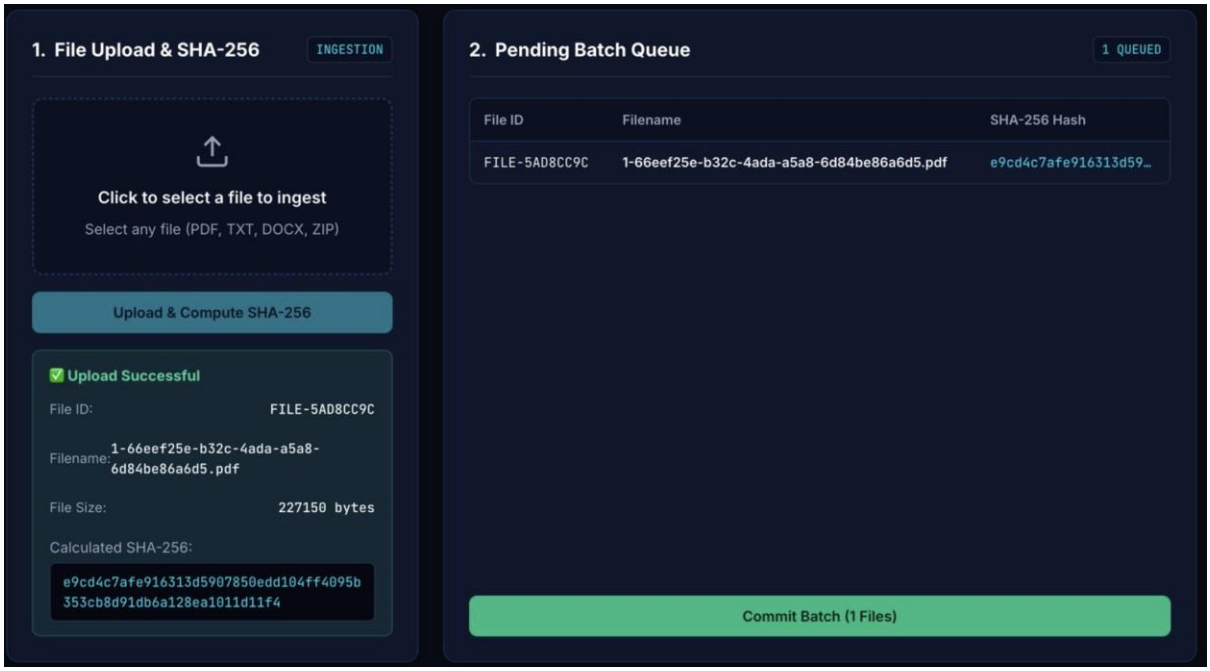
APPENDIX C — SCREENSHOTS



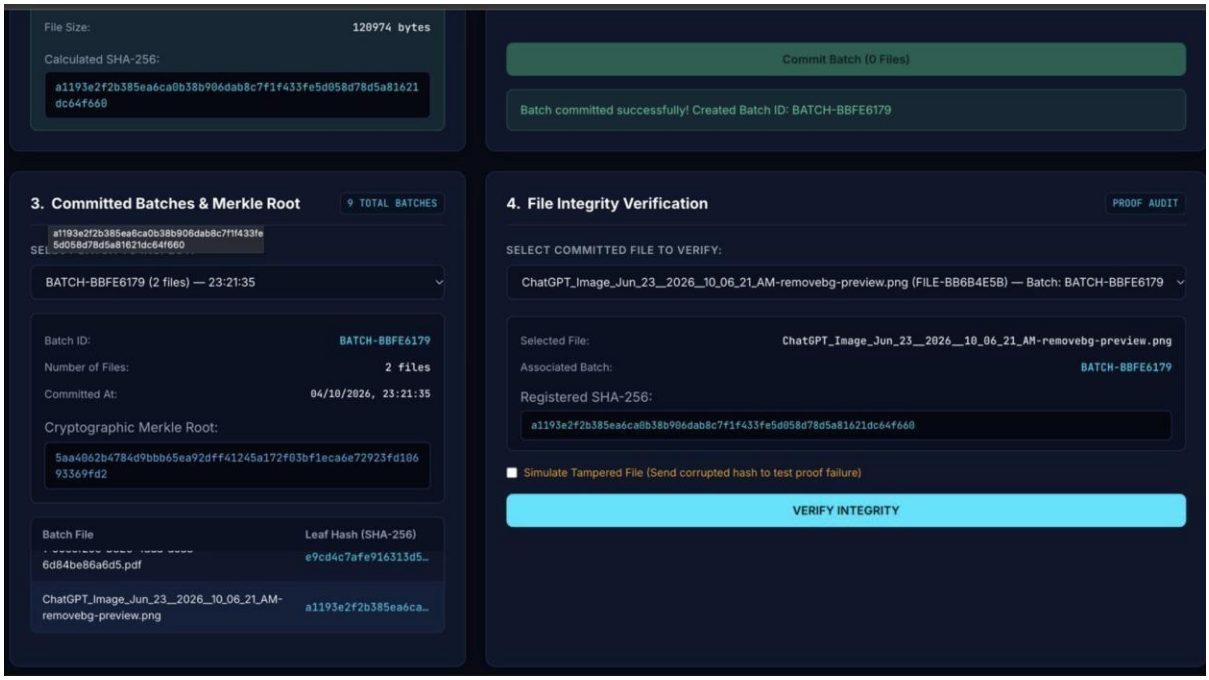
User interface



File upload



Batch committed

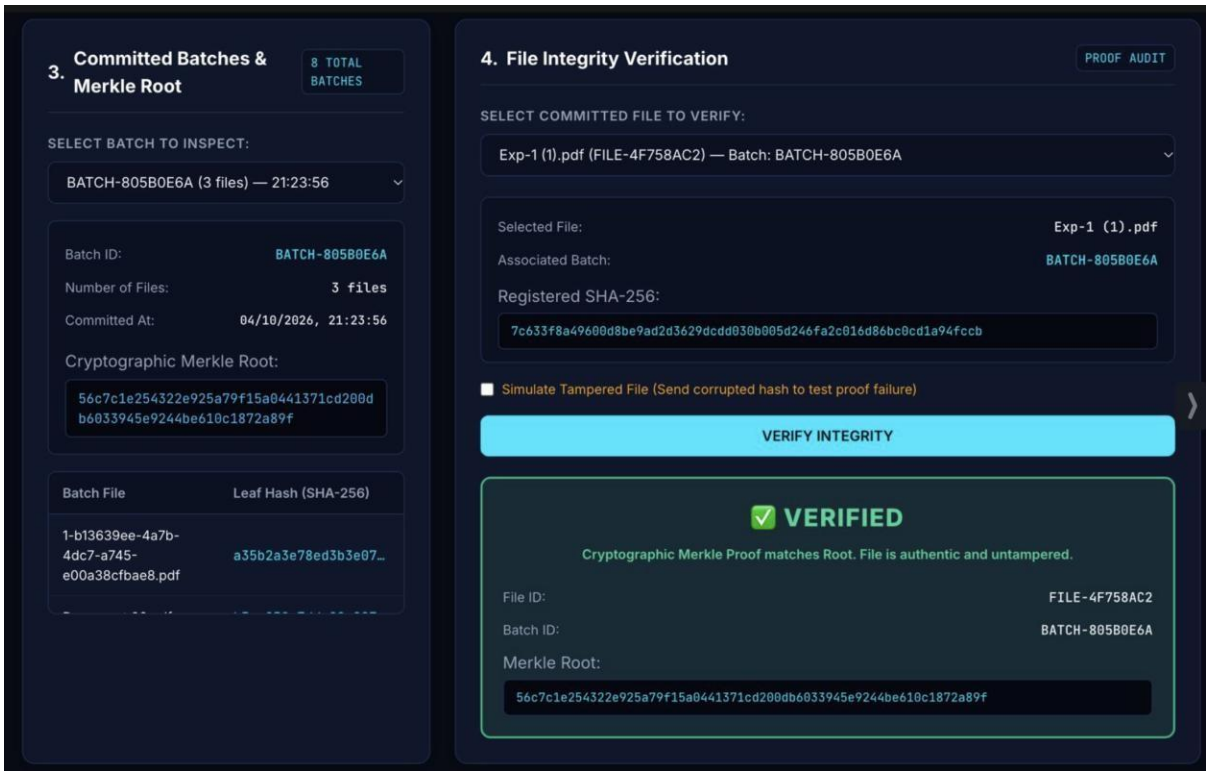




Select file to Verify



Verification



Simulation for tampered file

